

Les entrées/sorties en java

Jonathan Lejeune

SRCS – Master 1 SAR 2023/2024

sources et images

- cours précédent de J. Sopena
- javadoc

Définition d'une entrée/sortie (Input/Output)

Interaction entre un processus et un support extérieur :

- Entrée (Input) : le processus reçoit des données depuis un support
- Sortie (Output) : le processus produit des données vers un support

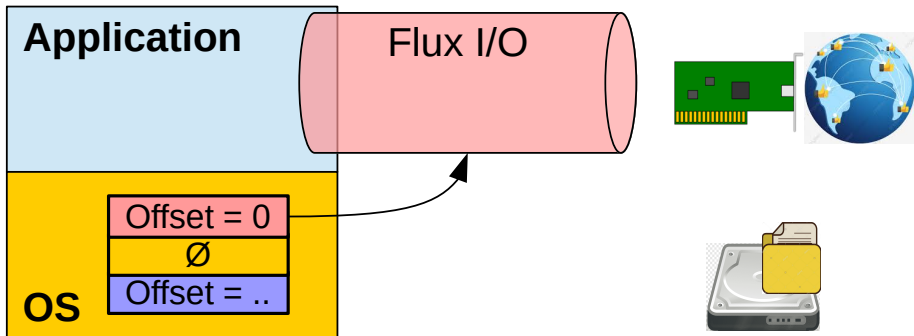
Exemples de supports

- **Entrée seulement** : clavier, souris, scanner, micro, webcam ...
- **Sortie seulement** : écran, imprimante, haut-parleurs ...
- **Entrée et Sortie** : stockage de masse (disque dur, clé USB), réseau, tampon mémoire

**Toute application informatique en production utilise
obligatoirement des I/O pour fonctionner**

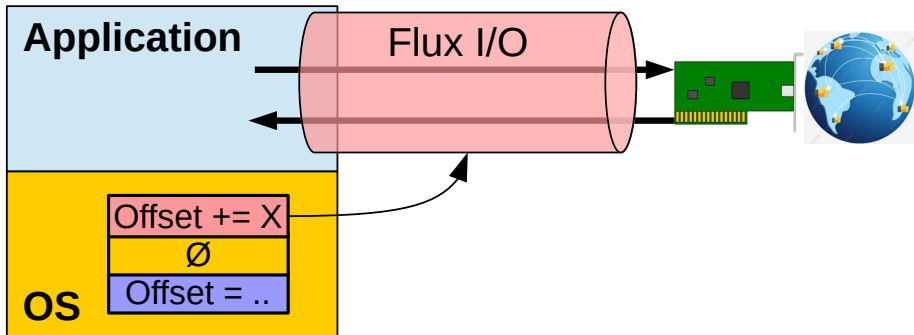
Étape 1 : Phase d'ouverture

- Indiquer à l'OS quel support on souhaite utiliser + paramétrage
- L'OS alloue une nouvelle ouverture dans une table interne et initialise éventuellement le support $\Rightarrow$ **maintien d'un offset**



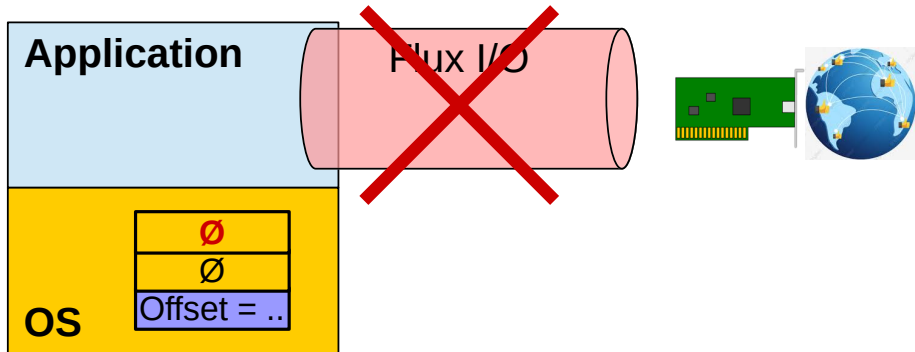
Étape 2 : Phase d'échanges de données

- Les données sont envoyées/reçues **séquentiellement** (octet par octet)
- L'OS incrémente l'offset à chaque échange



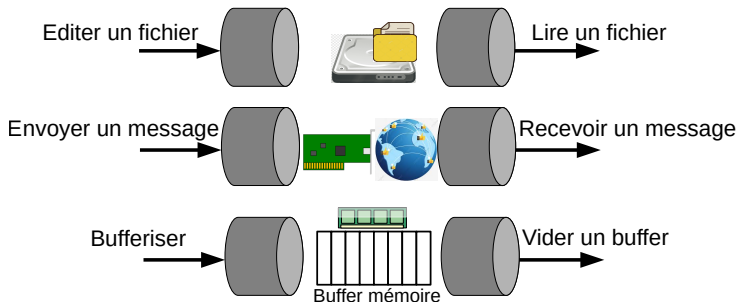
Étape 3 : Phase de fermeture

- Indiquer à l'OS que l'on ne souhaite plus utiliser l'ouverture
- L'OS libère l'entrée dans la table interne





Comment prendre en compte la diversité des actions sur des types de supports différents ?



Solution

Définition d'une abstraction des interactions

Objectif

Échanger des données sans se préoccuper du type du support.

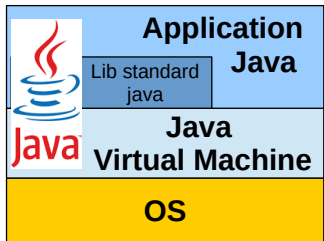
Abstraction offerte par UNIX

- Identifier n'importe quel support
⇒ tout support est identifiable par un lien du système de fichiers
- Créer un flux (**ouverture**)
⇒ Appel système `open`
- Identifier un flux/une ouverture
⇒ Descripteur de fichier (entier)
- Lire ou écrire des données dans un flux
⇒ Appels systèmes `read` et `write`
- Fermer un flux (**fermeture**)
⇒ Appel système `close`

Rappels

Un programme java :

- s'implémente en se basant sur une librairie standard de Java (JDK)
- s'exécute dans le cadre d'un environnement d'exécution Java (JVM)



Conséquences sur les I/O

- Le JDK redéfinit sa propre API des I/O avec un paradigme objet
⇒ package `java.io`
- La JVM fait office d'intermédiaire entre l'application java et l'OS
⇒ **Spécifications de l'API de l'OS** **possiblement modifiées**

Identification des flux

Un flux est une instance d'objet respectant un type abstrait de flux :

- `InputStream` : flux d'entrée d'octets → lire des `byte`
- `OutputStream` : flux de sortie d'octets → écrire des `byte`
- `Reader` : flux d'entrée de caractères → lire des `char`
- `Writer` : flux de sortie de caractères → écrire des `char`

Ouvrir un flux

- Ouvrir un flux = instancier une classe concrète de flux
- Une classe concrète est propre à un type de support
⇒ fichier, tableau d'octets, chaîne, tube, socket
- Les noms des classes concrètes respectent un format mnémotechnique

<TypeSupport><TypeAbstraitFlux>

ex : `FileInputStream`, `FileOutputStream`, `FileReader`, `FileWriter`

Primitives de lecture de InputStream et Reader

- `int read()` : lire un octet ou un caractère
 - **bloquant** tant qu'il n'y a rien à lire et que le flux reste ouvert.
 - l'octet/le caractère lu est retourné sous forme d'un `int`
 - retourne -1 si le flux est fermé
- `int read(byte[] tab)` OU `int read(char[] tab)` : lecture d'un tableau
 - **bloquant** tant qu'il n'y a rien à lire et que le flux reste ouvert.
 - retourne le nombre d'octets lus ou -1 si le flux est fermé
 - **Attention** : le tableau doit être alloué au préalable
- `int read(byte[] t, int o, int l)` OU `int read(char t[], int o, int l)` :
 - idem avec au plus l octets/caractères à partir de la case o

Autres primitives offertes

- `long skip(long n)` : parcourt n octets/caractères sans les retourner
- `int available()` : nombre d'octets disponibles (InputStream)

Primitives d'écriture de OutputStream et Writer

- `void write(int b)` : écrire un octet/un caractère sous un `int`
 - On ignore les 24 bits de poids fort pour les OutputStream
 - On ignore les 16 bits de poids fort pour les Writer
- `void write(byte b[])` / `void write(char cbuf[])` : écriture d'un tableau
- `void write(byte b[], int o, int l)` / `void write(char b[], int o, int l)`
 - écriture de `l` octets/caractères à partir de la case `o`
- `void write(String str)` : écrire une chaîne (spécifique aux writers)

Autre primitive offerte

- `void flush()` : vide les buffers en forçant l'écriture vers le support.

Fermeture du flux

Toute classe abstraite de flux hérite de l'interface `Closeable` qui offre la méthode `void close()`.

- Toute ouverture doit avoir une fermeture.
- Fermer un flux déjà fermé n'a aucun effet.
- Toute lecture/écriture à un flux fermé jette une `IOException`.

La classe IOException

- Classe racine de toutes les exception liées aux entrées/sorties
- **Attention** : ce n'est pas une RuntimeException
 - ⇒ directive `throws` ou bloc `try-catch-finally` obligatoire
- Tous les primitives et constructeurs des classes de flux sont déclarés avec `throws IOException`

Exemples de type d'exception

- `EOFException` : fin de flux inattendu lors d'une lecture
- `FileNotFoundException` : fichier introuvable
- `SocketException` : erreur lors de la création ou l'accès à une socket
- `RemoteException` : erreur lors d'un appel distant RMI

Schéma de l'utilisation d'un flux d'entrée sortie

Version classique

```
TypeFlux1 flux1;  
TypeFlux1 flux2;  
try{  
    flux1 = new FluxConcret1();  
    flux2 = new FluxConcret2();  
    //read ou write sur 1 et 2  
}catch(IOException e){  
    //do something  
}finally{  
    try{  
        flux1.close()  
        flux2.close()  
    }catch(IOException e){}  
}
```

Version avec AutoCloseable

```
try(TypeFlux1 flux1 = new  
    FluxConcret1();  
    TypeFlux2 flux2 = new  
        FluxConcret2()){  
    //read ou write sur 1 et 2  
}catch(IOException e){  
    //do something  
} //close implicite
```

Exemple d'utilisation des flux d'octets

Écriture

```
// Récupération du flux
OutputStream os = ?????? ;
// Utilisation du flux
for (byte b=0; b<5 ;b++) {
    os.write(b);
    os.write(2*b+1);
}
// Fermeture du flux
os.close();
```

Lecture

```
// Récupération du flux
InputStream is = ?????? ;
// Utilisation du flux
int x,y ;
while ((x=is.read())!=-1) {
    y = is.read();
    dessin.trace(x,y);
}
// Fermeture du flux
is.close();
```

Exemple d'utilisation des flux de caractères

Écriture

```
// Récupération d'un flux
Writer w = ?????? ;
// Utilisation du flux
w.write('a');
w.write('b');
w.write("cdef");
// Fermeture du flux
w.close();
```

Lecture

```
// Récupération d'un flux
Reader r = ?????? ;
// Utilisation du flux
StringBuffer buff = ...
int i;
while ((i=r.read())!=-1) {
    buff.append((char)i);
}
// Fermeture du flux
r.close();
```

Remarque importante

Les procédures d'écriture et de lecture doivent respecter le même protocole

Obtenir une référence sur un flux

Flux existant

- Soit à partir d'un accesseur sur un objet pouvant fournir un flux
ex : `getInputStream()`, `getOutputStream()`, ...
- Soit en accès direct sur un attribut public
ex : `System.in`, `System.out`, `System.err`

Créer un nouveau flux

Toute construction d'un objet héritant de `InputStream`, `OutputStream`, `Reader` ou `Writer`

Remarque

Le type de la référence du flux doit être le plus abstrait possible pour profiter du polymorphisme d'héritage

Classes permettant de construire des flux

Flux d'octets

Support	InputStream	OutputStream
Tableau en mémoire	ByteArrayInputStream	ByteArrayOutputStream
Fichier	FileInputStream	FileOutputStream
Pipeline entre threads	PipedInputStream	PipedOutputStream
Chaîne de caractères	StringBufferInputStream	StringBufferOutputStream

Flux de caractères

Support	Reader	Writer
Tableau en mémoire	CharArrayReader	CharArrayWriter
Fichier	FileReader	FileWriter
Pipeline entre threads	PipedReader	PipedWriter
Chaîne de caractères	StringReader	StringWriter

Objectif

Pouvoir ajouter dynamiquement des fonctionnalités ou changer les spécifications d'une primitive de flux

Application du design pattern décorateur

- Types abstraits à décorer : `Reader`, `Writer`, `InputStream`, `OutputStream`
- Types des décorateurs abstraits : `FilterReader`, `FilterWriter`, `FilterInputStream`, `FilterOutputStream`
 - ⇒ Ils héritent tous de leur flux de base respectif
 - ⇒ Ils ont tous une référence vers un flux renseignée au moment de leur construction
- Ils peuvent :
 - redéfinir le fonctionnement des méthodes de base (`read`, `write`, ...)
 - ajouter de nouvelles méthodes (`print`, `readline`, ...)

Les décorateurs fournis par java

Préfixe	Fonctionnalité	InputStream OutputStream Reader Writer			
Buffered :	Mise en tampon	✓	✓	✓	✓
Data :	Conversion des types primitifs	✓	✓	✗	✗
Object :	Sérialisation d'objet	✓	✓	✗	✗
Sequence :	Concaténation de flux	✓	✗	✗	✗
Print :	Fonctions de formatage print	✗	✓	✗	✓
PushBack :	Lecture avec remise dans le flux	✓	✗	✓	✗
LineNumber :	Numérotation des lignes	✓	✗	✓	✗
Digest :	Vérification par somme de contrôle	✓	✓	✗	✗
Cipher :	Chiffage du flux	✓	✓	✗	✗

Composition de décorateurs



```
OutputStream os = ?????? ;
```

```
for (byte b=0 ; b<3 ; b++) {  
    os.write(b);  
}  
os.close();
```

```
InputStream is = ?????? ;
```

```
while ( (x = is.read()) != -1 ) {  
    System.out.println(x);  
}  
is.close();
```

Composition de décorateurs



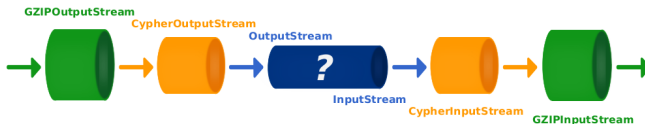
```
OutputStream os = ?????? ;  
CipherOutputStream cos = new CipherOutputStream(os, cipher);
```

```
for (byte b=0 ; b<3 ; b++) {  
    cos.write(b);  
}  
cos.close();
```

```
InputStream is = ?????? ;  
CipherInputStream cis = new CipherInputStream(is, cipher);
```

```
while ( (x = cis.read()) != -1 ) {  
    System.out.println(x);  
}  
cis.close();
```

Composition de décorateurs



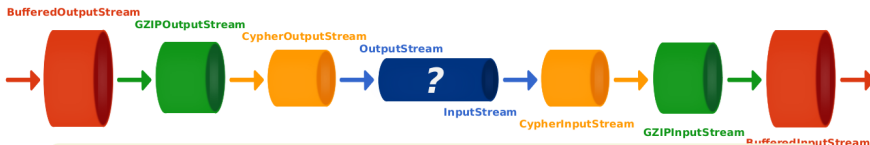
```
OutputStream os = ?????? ;  
CipherOutputStream cos = new CipherOutputStream(os, cipher);  
GZIPOutputStream zcos = new GZIPOutputStream(cos);
```

```
for (byte b=0 ; b<3 ; b++) {  
    zcos.write(b);  
}  
zcos.close();
```

```
InputStream is = ?????? ;  
CipherInputStream cis = new CipherInputStream(is, cipher);  
GZIPInputStream zcis = new GZIPInputStream(cis);
```

```
while ( (x = zcis.read()) != -1 ) {  
    System.out.println(x);  
}  
zcis.close();
```

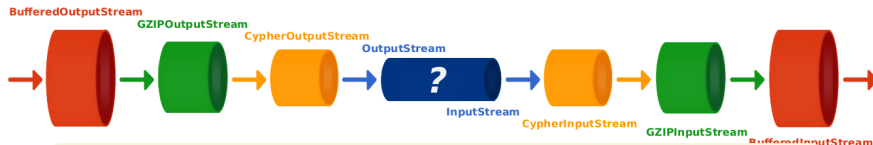
Composition de décorateurs



```
OutputStream os = ?????? ;
CipherOutputStream cos = new CipherOutputStream(os, cipher);
GZIPOutputStream zcos = new GZIPOutputStream(cos);
BufferedOutputStream bzcis = new BufferedOutputStream(zcos);
for (byte b=0 ; b<3 ; b++) {
    bzcis.write(b);
}
bzcis.close();
```

```
InputStream is = ?????? ;
CipherInputStream cis = new CipherInputStream(is, cipher);
GZIPInputStream zcis = new GZIPInputStream(cis);
BufferedInputStream bzcis = new BufferedInputStream(zcis);
while ( (x = bzcis.read()) != -1 ) {
    System.out.println(x);
}
bzcis.close();
```


Composition de décorateurs

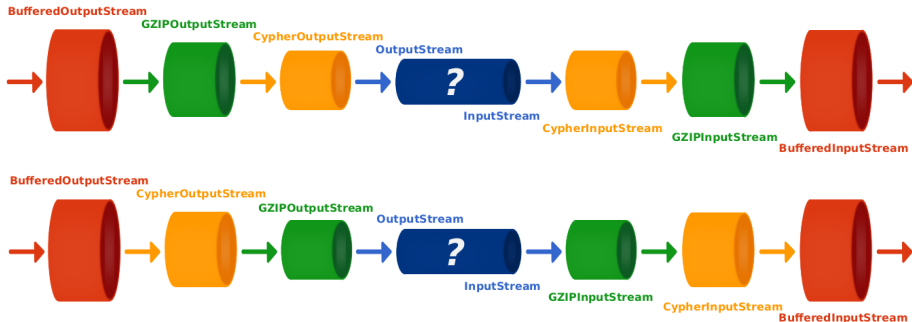


```
OutputStream os = new BufferedOutputStream(  
    new GZIPOutputStream(  
        new CypherOutputStream(??????, cipher)  
    ));  
for (byte b=0 ; b<3 ; b++) {  
    os.write(b);  
}  
os.close();  
  
InputStream is = new BufferedInputStream(  
    new GZIPInputStream(  
        new CypherInputStream(??????, cipher)  
    ));  
while ( (x = is.read()) != -1 ) {  
    System.out.println(x);  
}  
is.close();
```

Pièges de l'ordre des décorateurs



Laquelle de ces deux séquences choisiriez-vous ?



Influence sur les performances

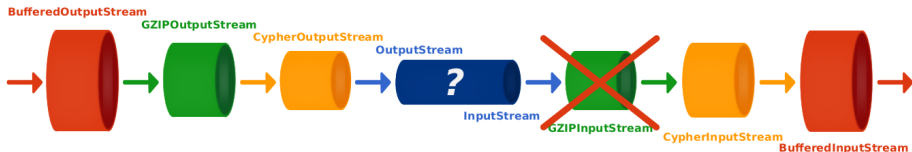
Les opérations de chiffrement sont très coûteuses en calculs :

⇒ il faut compresser avant de chiffrer

Pièges de l'ordre des décorateurs

Respecter la cohérence

L'ordre utilisé lors de la lecture d'un flux doit être compatible avec celui utilisé pour produire les données dans le flux de sortie



```
java Reception
Exception in thread "main" java.util.zip.ZipException: Not in
GZIP format
    at java.util.zip.GZIPInputStream.readHeader(GZIPInputStream.java:164)
    at java.util.zip.GZIPInputStream.<init>(GZIPInputStream.java:78)
    at java.util.zip.GZIPInputStream.<init>(GZIPInputStream.java:90)
    at flux.FluxBZC.main(Reception.java:10)
```

Définir ses propres décorateurs de filtre

Marche à suivre

Pour un type abstrait de flux `X` :

- 1 Définir une classe qui hérite du décorateur correspondant `FilterX`
- 2 Implémenter un constructeur prenant en paramètre un `X x`
 - 2.1 appeler à la première ligne du constructeur `super(x)`;
 - 2.2 si besoin enregistrer les autres paramètres dans des attributs
- 3.a Si `X` est un flux de lecture, redéfinir la méthode `int read()` :
 - 3.a.1 Appeler `super.read()` pour obtenir la valeur avant transformation
 - 3.a.2 Appliquer la nouvelle fonctionnalité
 - 3.a.3 Retourner la valeur modifiée
- 3.b Si `X` est un flux d'écriture, redéfinir la méthode `write(int d)` :
 - 3.b.1 Appliquer la nouvelle fonctionnalité sur `d`
 - 3.b.2 Appeler `super.write(..)` pour écrire après transformation

Attention

S'assurer que toute surcharge de `read` et `write` appelle votre méthode

Exemple de définition de décorateur : fonction linéaire

```
public class LinearInputStream extends FilterInputStream {
    private final int a,b;
    public LinearInputStream(InputStream is,int a, int b) {
        super(is);
        this.a=a;
        this.b=b;
    }
    @Override
    public int read() throws IOException {
        int x = super.read() ;
        return a*x+b ;
    }
}
```

```
OutputStream os =
    new FileOutputStream("f1");
os.write(1);
os.close();
```

```
InputStream is =
    new FileInputStream("f1");
LinearInputStream lis =
    new LinearInputStream(is,2,3);
int y = lis.read();
System.out.println("y="+y);
lis.close();
```